

Article

A Lightweight, End-to-End Encrypted Data Pipeline for IIoT: An AES-GCM Implementation for ESP32, MQTT, and Raspberry Pi

Gulshat Amirkhanova ¹, Syrym Ismailov ^{1,*}, Alikhan Amirkhanov ¹, Saltanat Adilzhanova ¹,
Meiramkul Zhasuzakova ² and Siming Chen ³

¹ Faculty of Information Technology and Artificial Intelligence, Al-Farabi Kazakh National University, Almaty 050040, Kazakhstan; gulshat.amirkhanova@kaznu.edu.kz (G.A.); amirkhanov_alikhan@kaznu.edu.kz (A.A.); adylzhanova.s@kaznu.edu.kz (S.A.)

² Faculty of Information Technologies and Artificial Intelligence, Almaty Technological University, Almaty 050012, Kazakhstan

³ School of Data Science, Fudan University, Shanghai 200437, China

* Correspondence: syrym.ismail@gmail.com

Abstract

Industrial Internet of Things (IIoT) deployments increasingly rely on low-cost microcontrollers and single-board computers to stream operational telemetry for monitoring, control, and predictive maintenance, yet the canonical “TLS-to-broker” model does not protect message content from a compromised or curious MQTT broker. This study therefore designs and implements a practical, application-layer end-to-end (E2E) encryption pipeline spanning an ESP32 data client (C++/mbedTLS), an untrusted MQTT broker, and a Raspberry Pi gateway (Python/PyCryptodome) using AES-256-GCM with Additional Authenticated Data (AAD). Sensor measurements are serialized as compact JSON, encrypted and authenticated on the ESP32, framed into a binary record, Base64-encoded for MQTT payload carriage, and verified/decrypted only at the gateway. Experiments on ESP32-WROOM-32 and Raspberry Pi 4 show an average ESP32 packet-preparation latency of 41.754 ms (JSON 1.0 ms; AES-GCM 29.5 ms; Base64 11.2 ms), robust rejection of ciphertext tampering and unauthorized devices via MAC verification and whitelist checks, and 99.72% decrypt-and-store success over a one-hour run (718/720 messages). These results indicate that commodity IIoT hardware can support practical and replicable E2E confidentiality and integrity without sacrificing operational throughput, while eliminating the MQTT broker as a de facto man-in-the-middle.

Keywords: Industrial IoT (IIoT); MQTT; security; end-to-end encryption; AES-GCM; ESP32; mbedtls; PyCryptodome



Academic Editor: Qiong Huang

Received: 18 November 2025

Revised: 18 December 2025

Accepted: 30 December 2025

Published: 3 January 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and

conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Industry 4.0 has accelerated the deployment of connected sensors and actuators across energy communities, buildings, transportation, and manufacturing sites, where publish/subscribe messaging and edge processing enable responsive, data-driven operations and cyber-physical control loops [1–7]. Representative implementations include demand-response testbeds for smart communities [1], low-cost embedded smart-city platforms [2], privacy-oriented open IoT infrastructures for research and industry [3], Raspberry Pi gateways bridging legacy equipment to dashboards [4], residential HVAC retrofits relying on Wi-Fi and MQTT [5], MQTT-based interaction frameworks in robotics [6], and smart-home

monitoring stacks that adopt transport-layer security [7]. Across such deployments, operational measurements may reveal equipment states, occupancy patterns, energy usage, safety-critical alarms, or protected industrial processes, making confidentiality and integrity essential rather than optional.

1.1. MQTT in IIoT: Operational Constraints and Security Limitations

MQTT is attractive for IIoT because it is lightweight, decouples publishers and subscribers, and supports QoS policies that can trade bandwidth for delivery guarantees. However, the broker is structurally central: it terminates sessions, enforces topic routing, can persist retained messages, and often hosts plugins, monitoring, and logging. This centrality is also the primary security challenge in practice. Empirical studies show that broker performance and resilience degrade under malformed traffic and SYN-flooding, and that higher QoS levels can amplify processing and transmission overheads on constrained broker hardware such as Raspberry Pi [8–10]. Even in benign conditions, transport-layer configuration matters: cipher negotiation, TLS versions, and handshake behavior measurably affect bandwidth and latency for constrained clients and brokers [11]. In recent V2X-oriented experiments, TLS-enhanced MQTT on IoT devices exhibited non-trivial CPU and energy costs during handshake, with performance varying by cipher suite (e.g., ChaCha20-Poly1305 sometimes outperforming AES-GCM in wireless scenarios) [12]. These findings motivate a distinction that is often blurred in deployments: while TLS protects the channel between client and broker, it does not prevent the broker itself from seeing or modifying plaintext because the broker is the endpoint of the TLS session.

This “TLS-to-broker” model is frequently acceptable when the broker operator is fully trusted, but it is misaligned with common Industry 4.0 realities in which brokers run in shared infrastructure, are managed by third parties, or are exposed to insider threats. Multiple application domains demonstrate valuable defense-in-depth with TLS and authenticated telemetry flows [13–18], yet those architectures continue to rely on the broker as a trusted intermediary for message content. In such settings, the broker becomes a “man-in-the-middle by design”: it is positioned to inspect, log, or tamper with measurements even if the network transport is encrypted. Consequently, practical E2E protection must be enforced at the application layer so that only endpoints can interpret and authenticate telemetry.

1.2. Related Work: Lightweight Cryptography and Application-Layer Protection

A substantial literature evaluates cryptographic choices and their resource impact on embedded devices. Comparative analyses show that algorithm selection affects latency, payload expansion, and power consumption across ESP-class microcontrollers and Raspberry Pi-class gateways [19]. Lightweight authenticated encryption approaches, including ASCON, can outperform AES-GCM on certain embedded platforms or workloads [20,21], whereas AES variants tailored for constrained devices may improve execution time or implementation characteristics on microcontrollers [22]. In IoMT/IoT authentication frameworks, symmetric primitives are often favored because of their lower compute and energy cost compared with public-key alternatives [23]. Domain-specific studies also report that lightweight ciphers (e.g., SPECK) may reduce execution time relative to AES in constrained GSM/MQTT environments, although algorithm choice must also consider standardization, interoperability, and threat models [24].

Beyond classical symmetric cryptography, post-quantum cryptographic mechanisms have been explored for future resilience. Measurements show that lattice-based KEMs can be expensive for low-end MCUs, particularly due to memory and vector arithmetic demands [25], while Raspberry Pi-class devices can host several PQC primitives within sub-100 ms operations in some implementations [26]. Hybrid IoMT designs combining

TinyML with post-quantum cryptography demonstrate feasibility primarily at gateway or network levels rather than on the smallest sensor nodes [27]. Hybrid and composite cryptosystems have also been proposed for strengthening IoT and 5G/B5G security at higher performance tiers [28]. In parallel, chaos-augmented designs have been studied in robotic and vehicular contexts, including ASCON-based constructions [29,30]. Numerous application-layer or on-device encryption approaches exist for specific domains, such as chaos-based image encryption on Raspberry Pi [15], LoRa image security schemes [31], DNA-computing stream ciphers over MQTT [32], and on-sensor stream-cipher encryption to protect smart-city measurements directly on constrained sensors [33]. These works reinforce a key design principle: protecting the payload object can be more robust than protecting only the transport channel, particularly under broker compromise.

Practical security building blocks on commodity hardware are similarly well documented. Educational security laboratories pair ESP32 platforms with cryptographic stacks such as mbedTLS and hardware add-ons [34]; device managers implement symmetric encryption and integrity checks for payloads and updates on ESP32 [35]; and ECC-backed authentication devices for multiprotocol environments report sub-second signing and verification on embedded platforms [36]. Commercial development boards also integrate secure elements to protect credentials and assist with secure onboarding [37]. Across predictive maintenance, agriculture, safety, and healthcare telemetry, MQTT remains a common spine for transporting time-series data to gateways and analytics platforms [38–45], including ESP32-CAM security alarms [46], public-safety and emergency response systems [17], fleet telemetry and control [18], secure vehicle voice chat using MQTT/CoAP [47], and HL7/MQTT medical image monitoring [48]. Edge gateways consolidate protocol mediation, storage, and local control [49], while device identity and provenance mechanisms can incorporate PUFs and blockchain [50]. Lightweight encryption and authentication continue to evolve via block-cipher designs [51], SRAM-PUF mutual authentication [52], cloud registration approaches [53], sensing pipelines with AI [54], and PUF-tied keyed hashing [55], highlighting a diverse design space for secure IIoT architectures.

1.3. Research Gap and Contributions

Within this landscape, a practical gap remains. Many systems secure transport links or propose novel ciphers, yet relatively few deliver a complete, reproducible, and benchmarked application-layer E2E pipeline that explicitly treats the MQTT broker as untrusted, particularly on a heterogeneous implementation stack where encryption is executed on an ESP32 (C++/mbedTLS) and decryption/authorization on a Raspberry Pi gateway (Python/PyCryptodome). Prior work provides important measurements of broker resilience, TLS overhead, or algorithm performance [8–12,19–24], but it less often combines (i) a concrete E2E framing protocol; (ii) explicit endpoint authorization bound to authenticated metadata; and (iii) microcontroller-side latency measurements together with adversarial validation (tamper rejection) and operational reliability in an integrated pipeline.

This paper addresses that gap by specifying, implementing, and experimentally evaluating a practical E2E encrypted IIoT data path using AES-256-GCM with associated data to provide confidentiality and integrity end-to-end while treating the broker purely as a relay. The core contributions are as follows. First, we define a compact AES-GCM framing format and reference implementation that is interoperable across a constrained ESP32 publisher and a Python gateway, enabling broker-agnostic E2E protection on commodity hardware. Second, we introduce device-level authorization bound to AAD and validate integrity under ciphertext tampering, demonstrating that modifications by a broker or network adversary are detected and rejected. Third, we provide reproducible microcontroller-side timing measurements for serialization, encryption, and encoding, and we evaluate end-to-end

reliability under sustained encrypted acquisition. Finally, we explicitly document a threat model, nonce/key management considerations, and limitations to clarify which attacks are mitigated and which require complementary controls.

Encryption is not a substitute for monitoring and intrusion detection; rather, it should be layered with anomaly detection and IDS mechanisms that can operate at gateways or network boundaries. Recent work on adversarially robust IoT intrusion detection highlights this complementary role, especially against evasion attacks that target ML-based IDS deployments [56].

2. Materials and Methods

2.1. System Architecture

The pipeline comprised three roles. The Data Client (ESP32-WROOM-32) sampled local sensor values, serialized them as compact JSON, performed AES-GCM encryption at the application layer, and published a Base64-encoded frame to an MQTT topic over Wi-Fi. The Message Broker handled publish/subscribe delivery but was explicitly considered untrusted with respect to plaintext; transport-layer protections (e.g., TLS) were treated as optional defense-in-depth rather than a trust anchor. The Gateway Server (Raspberry Pi 4) subscribed to the topic, verified device authorization via a whitelist bound to the Additional Authenticated Data (AAD), authenticated and decrypted packets, and stored accepted measurements (e.g., in InfluxDB) for downstream analytics [4,24].

Figure 1 illustrates the proposed architecture and data flow, emphasizing that encryption and authentication occur only at the endpoints.

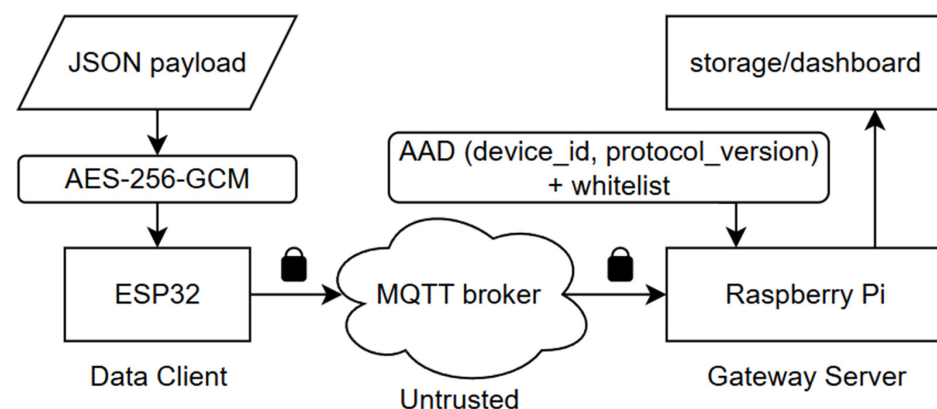


Figure 1. Proposed E2E secure IIoT data pipeline architecture, showing encrypted payload flow from ESP32 client to Raspberry Pi gateway via an untrusted MQTT broker.

Because the broker is assumed untrusted, the system is designed so that compromise of the broker yields only ciphertext and authenticated metadata, not plaintext measurements. The gateway serves as the first trusted termination point for measurement content and is the appropriate location for storage, access control, and optional anomaly detection modules [49].

2.2. Hardware and Software Stack

We implemented the client on an ESP32-WROOM-32 development board, which provides sufficient compute for authenticated encryption under typical IIoT rates while maintaining a small energy and memory footprint [23,36]. A DHT22/AM2302-class sensor served as a representative low-rate telemetry source [57]. The gateway ran on a Raspberry Pi 4, representative of low-cost edge servers used in IIoT deployments [1,4,12]. MQTT connectivity used Eclipse Paho clients; the broker was Eclipse Mosquitto. The client used mbedTLS (C/C++) for AES-GCM and Base64; the gateway used PyCryptodome (Python)

for AES-GCM and paho-mqtt for messaging. An optional time-series backend (InfluxDB) and visualization stack may be used for persistence and dashboarding, consistent with open IIoT platform patterns [3,24,38].

2.3. Secure Data Protocol Design

2.3.1. Plaintext Payload and AAD

The plaintext payload was serialized as compact JSON with fields such as `device_id`, `ts` (timestamp), and a measurement bundle (e.g., voltage, current, temperature, humidity). The `device_id` string and a fixed protocol version were used as Additional Authenticated Data (AAD) so that critical metadata remains visible for routing and authorization but is cryptographically bound to the ciphertext. This prevents an adversary from selectively altering device identifiers or protocol markers without triggering authentication failure.

2.3.2. Cryptographic Parameters and Design Rationale

Keys were 256-bit pre-shared keys (PSKs) provisioned out-of-band for prototype simplicity; symmetric keys are computationally efficient on constrained nodes relative to more complex asymmetric mechanisms [23]. We selected a 12-byte nonce and 16-byte tag consistent with standard AES-GCM practice, balancing performance and integrity strength. The 12-byte nonce enables efficient GCM processing, while a 16-byte tag provides strong message authenticity against forgery under standard threat assumptions. AAD is used to bind metadata to ciphertext without expanding the encrypted payload, and it enables explicit device authorization at the gateway.

A compact binary frame was Base64-encoded for MQTT transport to improve interoperability with common broker tooling and message inspection utilities; Base64 incurs overhead but provides reliable ASCII carriage and simplifies debugging. In bandwidth-critical deployments, the binary frame can be transmitted directly as a binary MQTT payload, avoiding Base64 overhead at the cost of reduced human readability.

2.3.3. Encryption Frame on ESP32 (C++/mbedTLS)

On the ESP32, the client generated a fresh nonce per message and executed `mbedtls_gcm_crypt_and_tag` to encrypt the plaintext and generate the tag in one pass. The client then constructed a compact frame:

[Version | Nonce (12 B) | Tag (16 B) | Ciphertext (N B)]

The frame was Base64-encoded and published as the MQTT payload. The design is endpoint-only: the broker sees only Base64 ciphertext and cannot reconstruct plaintext or modify the encrypted content without detection.

2.3.4. Decryption Frame on Raspberry Pi (Python/PyCryptodome)

On the Raspberry Pi gateway, the subscriber decoded Base64, parsed the fixed-width header to recover version, nonce, and tag, and then initialized `AES.new(key, AES.MODE_GCM, nonce = nonce)` in PyCryptodome. The gateway provided the same AAD and executed `cipher.decrypt_and_verify(ciphertext, tag)`. Successful verification yielded authenticated plaintext JSON, which was parsed, authorized against a `device_id` whitelist, and written to storage. Failures at MAC verification or authorization produced structured logs and dropped the packet without side effects.

2.3.5. Nonce Management, Reboot Behavior, and Replay Considerations

The security of AES-GCM depends critically on nonce uniqueness per key. In our prototype, a new nonce is generated per message using the device's cryptographic random-

ness at runtime, and the gateway rejects messages failing authentication. For long-lived deployments, additional engineering controls are recommended to further reduce the risk of nonce reuse after reboot, counter wrap-around, or firmware update events. A robust approach is to generate a boot-unique prefix at startup and concatenate it with a monotonic counter, persisting the counter in non-volatile storage; this provides deterministic uniqueness across resets. Replay defenses can additionally be implemented by incorporating a sequence number or timestamp into the authenticated plaintext and maintaining a per-device acceptance window at the gateway. These considerations are discussed further in Section 4 to clarify operational assumptions and limitations.

2.4. Experimental Design

We assume an adversary who can observe network traffic, compromise or operate the MQTT broker, and attempt message injection, tampering, or replay. Under this model, the broker may log or modify payloads, and a network attacker may perform man-in-the-middle manipulation of MQTT traffic. We also consider denial-of-service conditions such as broker flooding and malformed MQTT traffic, which are observed to degrade MQTT infrastructure in practice [8–10]. The primary security goals are: confidentiality of measurements against broker/network observers; integrity and authenticity of measurements so that tampering is detected; and device authorization so that only registered devices may write to storage.

The threat model necessarily includes assumptions. In particular, endpoint compromise is difficult to prevent by cryptography alone. This work does not claim to “prove” the ESP32 is trusted; rather, it treats the endpoint firmware and secret key material as part of the trusted computing base. Physical compromise, key extraction, and side-channel attacks (timing, power, EM) are acknowledged as important risks but are outside the scope of the presented evaluation. Practical mitigations include secure boot and flash encryption on ESP-class devices, disabling debug interfaces, using secure elements for credential protection [37], and hardware-bound identity mechanisms such as PUF-based designs [50,52,55]. These mitigations complement—but do not replace—E2E cryptography.

2.5. Experimental Design and Benchmark Scenarios

Four evaluations were conducted to validate feasibility and operational impact. First, computational overhead on the ESP32 was quantified by timing JSON serialization, AES-GCM encryption + tagging, and Base64 encoding across 1000 iterations using microsecond-resolution timers, reporting average, minimum, and maximum latencies and the mean packet-preparation cost. Second, security and integrity were validated by inducing ciphertext tampering (single-byte flips) and device authorization violations (non-whitelisted device_id), confirming that the gateway rejected such packets with explicit MAC-failure and authorization-failure logs. Third, end-to-end throughput and reliability were assessed by running a one-hour test at one message every five seconds (720 total), recording the number of successfully decrypted and stored messages to compute packet success rate. Fourth, the system behavior was qualitatively assessed under typical broker conditions to ensure that encryption did not destabilize the MQTT publish/subscribe pipeline.

Figure 2 summarizes the overall implementation workflow of the proposed end-to-end encrypted IIoT pipeline, from system scoping and firmware development to broker deployment, gateway processing, and end-to-end validation.

The benchmark scope is intentionally focused: a single ESP32 board, a single gateway, a low-rate sensor payload, and a single message rate in the long-run test. These constraints are made explicit to avoid overgeneralization, and Section 4 discusses expected scaling behavior

and additional experiments needed to quantify performance across higher message rates, varying payload sizes, and multiple concurrent publishers.

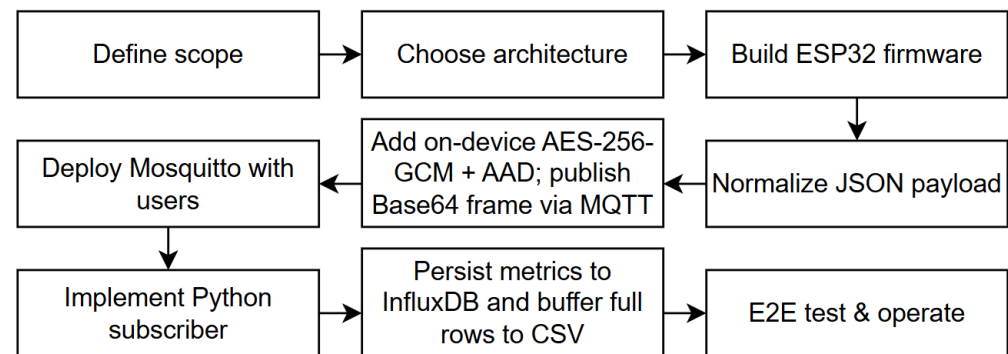


Figure 2. Implementation workflow for the end-to-end encrypted IIoT data pipeline, from scope definition and firmware development through broker/gateway deployment and end-to-end validation.

3. Results

3.1. ESP32 Packet-Preparation Latency

Packet-preparation latency on the ESP32 was characterized by instrumenting three code regions—JSON serialization, AES-GCM encryption and tag generation, and Base64 encoding—over 1000 consecutive messages. The measured overall mean latency was 41.754 ms per packet. Table 1 summarizes the component costs: JSON serialization contributed 1.0 ms on average (0.8–1.2 ms), AES-GCM encryption and authentication dominated at 29.5 ms on average (27.0–32.0 ms), and Base64 encoding added 11.2 ms on average (10.0–13.0 ms). The sum of component means (41.7 ms) closely matches the measured end-to-end mean, with the small residual attributable to measurement overhead and timing granularity.

Table 1. ESP32 packet-preparation latency (1000-packet run).

Component	Avg Latency (ms)	Min (ms)	Max (ms)
JSON serialization	1.0	0.8	1.2
AES-GCM encrypt + tag	29.5	27.0	32.0
Base64 encoding	11.2	10.0	13.0
Total (measured)	41.754		

At the operating regime used in the one-hour reliability test—one message every five seconds—the preparation latency consumes approximately 0.84% of the duty cycle (41.754 ms of 5000 ms), leaving substantial headroom for sensing, networking, and application logic. Because the encryption/encoding operations are per-message and do not depend on the number of prior packets, increasing the loop length from 1000 to 5000 iterations is not expected to increase per-packet mean latency; rather, it primarily improves the stability of the estimated mean and variability. These findings align with broader evidence that symmetric authenticated encryption remains practical on ESP-class devices relative to more complex asymmetric or post-quantum alternatives at the sensor tier [19,23,25–27].

3.2. Security and Integrity Validation

The correctness of AES-GCM confidentiality and integrity protection, together with device-whitelisting, was validated by exercising three scenarios against the Raspberry Pi gateway while capturing representative log output (Table 2). For a valid message from a whitelisted device, the gateway decoded Base64, verified the AES-GCM tag with the configured AAD, parsed authenticated plaintext JSON, and wrote measurements to InfluxDB. For

the ciphertext-tampering scenario, an intercepted payload was decoded, a single byte was altered in the ciphertext segment, and the modified frame was republished. The gateway rejected the tampered message, reporting a MAC verification failure. This demonstrates that the broker (or any network attacker) cannot modify payload bytes without detection, directly addressing the broker-tampering concern.

Table 2. Representative gateway logs under valid, tampered, and unauthorized cases (with English gloss).

Scenario	Example Log Line
Valid message	2025-11-14 14:22:11,184—INFO—[ESP32_Dala_Meter_001989] запись в InfluxDB—OK (write to InfluxDB—OK)
Tampered frame	2025-11-14 14:22:16,321—ERROR—Ошибка расшифровки/валидации кадра: AES-GCM decrypt/verify failed: MAC check failed (decrypt/verify failed: MAC check failed)
Unauthorized device	2025-11-14 14:22:21,907—WARNING—Device ‘ESP32_Test_9999’ не в devices.txt—сообщение проигнорировано. (device not in whitelist—message ignored)

In the unauthorized-device scenario, a syntactically valid AES-GCM frame was generated with a device_id absent from the local whitelist. Even if cryptographic verification succeeds, the gateway enforces authorization policy by rejecting unregistered devices, thereby preventing silent data injection into storage without administrative enrollment.

3.3. End-to-End Throughput and Reliability

End-to-end reliability was assessed by sending one encrypted message every five seconds for one hour, yielding an expected total of 720 messages. Gateway logs and InfluxDB records showed that 718 messages were successfully decrypted and stored, corresponding to a packet success rate of 99.72% (Table 3). Two messages were not committed to storage over the one-hour window. Because no cryptographic verification failures were observed during the soak test, these losses are consistent with transient network or broker/gateway processing conditions rather than integrity failures; nonetheless, the experiment did not instrument root-cause telemetry such as broker queue depth or Wi-Fi RSSI at the moment of loss.

Table 3. End-to-end reliability during a 1 h encrypted acquisition test.

Metric	Value
Total messages sent	720
Successfully stored	718
Packet success rate (%)	99.72

From an operational perspective, loss of 2 packets in 720 at this sampling rate is unlikely to affect typical monitoring workloads, particularly for slowly varying process variables. For applications requiring stronger delivery guarantees, reliability can be further improved through MQTT QoS policies, persistent sessions, and application-level acknowledgements or retransmission strategies, as discussed in Section 4.

4. Discussion

4.1. Practical Contribution Beyond TLS-Only MQTT

The results substantiate that application-layer E2E encryption based on AES-GCM is compatible with real-time telemetry on commodity IIoT hardware. The ESP32 packet-preparation latency of 41.754 ms is modest relative to the 5 s sampling interval used in the long-run test, and it is comparable to or smaller than the overheads already tolerated

in TLS-secured MQTT deployments, where handshake, cipher-suite selection, and wireless conditions can introduce tens to hundreds of milliseconds of latency and non-trivial CPU/energy costs [11,12]. The key difference is architectural: TLS protects the client-to-broker channel, whereas the proposed approach protects message content end-to-end and remains secure even if the broker is compromised or operated by an untrusted party. This distinction is particularly important when broker resilience and security are known to be stress points in practice, including under malformed traffic or denial-of-service conditions [8–10].

4.2. Comparison with Lightweight AEAD and Object-Security Approaches

The literature highlights multiple candidate strategies for protecting IoT payloads. Some approaches rely on transport security and broker hardening, while others adopt application-layer encryption tailored to domain constraints [13–16,31–33]. Our design falls into the latter category, using a widely supported AEAD primitive (AES-GCM) to maximize interoperability across embedded C/C++ stacks and Python gateway environments.

To clarify how AES-GCM relates to other lightweight approaches, Table 4 provides a focused qualitative comparison grounded in the cited literature. ASCON-based solutions, including chaos-augmented variants, can offer attractive performance profiles on some constrained platforms [20,21,29,30]. TLS evaluations also underscore that cipher-suite choice (e.g., ChaCha20-Poly1305 vs. AES-GCM) can matter in wireless settings [12]. Stream-cipher approaches implemented directly on sensors can reduce overhead in some traffic patterns and simplify on-device integration [33]. In the present work, AES-GCM was selected because it provides standardized AEAD security, has mature implementations, and can be deployed consistently across ESP32 and Raspberry Pi using widely adopted libraries.

Table 4. Qualitative comparison of candidate payload-protection approaches for constrained IIoT.

Approach	Typical Strengths	Typical Constraints	Example Works
AES-GCM (application-layer AEAD)	Mature standard; strong integrity; broad library support across C/C++ and Python; interoperable framing	Requires strict nonce uniqueness; performance may be heavier than some lightweight AEADs	This work; algorithm evaluations in [19,22]
ASCON/lightweight AEAD	Designed for lightweight profiles; can outperform AES-GCM on some embedded targets	Not always available in standard embedded stacks; interoperability/tooling less uniform	[20,21,29,30]
TLS-secured MQTT (channel security)	Strong channel protection; operationally familiar	Broker still sees plaintext; handshake/CPU/energy overhead can be significant; broker compromise remains critical	[11–18]
Domain-specific stream ciphers/specialized payload encryption	Can be lightweight and on-sensor; payload protected independent of channel	Often domain-specific; may reduce interoperability; still needs key/nonce discipline	[32,33]

This comparison motivates an interpretation of the present results as a practical baseline. Rather than claiming AES-GCM is always optimal, the contribution is to demonstrate that a standardized AEAD can be implemented end-to-end on commodity hardware with explicit measurements and adversarial validation, and that the pipeline can serve as a reference for future evaluations of ASCON or other AEAD candidates in the same architecture.

4.3. Security Beyond Broker Compromise: Replay, DoS, and Side Channels

While the primary motivation is broker compromise, IIoT deployments face broader threats. Message replay and injection are natural concerns in publish/subscribe systems.

AES-GCM integrity ensures that payload modifications are detected, as validated by the MAC failure behavior in Table 2; however, replay protection requires an additional policy layer, typically via sequence numbers or timestamps authenticated in the payload. Because the plaintext JSON includes a timestamp *ts*, gateway-side replay defenses can be implemented as a per-device acceptance window without exposing plaintext to the broker. Denial-of-service threats remain relevant even with E2E encryption because availability is not guaranteed by cryptography. Broker-level stress under malformed traffic and flooding has been observed experimentally [8–10], reinforcing the need for operational controls such as rate limiting, topic ACLs, and monitoring.

Timing, power, or EM leakage can threaten key material on constrained endpoints. This work does not attempt side-channel resistance validation, and it assumes the endpoint key is not extracted. Mitigations in realistic deployments include secure boot and flash encryption, disabling debug interfaces, using secure elements for key storage [37], and adopting hardware-bound identity and authentication approaches (e.g., PUF-based schemes) [50,52,55]. These measures help address the risk that a compromised device could leak keys and thereby undermine E2E encryption.

4.4. Key Management, Provisioning, Rotation, and Revocation at Scale

The prototype uses pre-shared 256-bit keys provisioned out-of-band to emphasize a simple and replicable baseline. This approach is common in small deployments, but it becomes limiting for large fleets because it complicates secure onboarding, key rotation, and revocation. Multiple directions are suggested by the literature: ECC-based authentication devices demonstrate practical signing and verification on embedded targets [36]; secure elements provide protected key storage and facilitate secure cloud onboarding workflows [37]; and PUF-based approaches can bind device identity to hardware characteristics and reduce exposure of long-term secrets [50,52,55]. Cloud registration workflows for IoT boards have also been explored [53]. In a production IIoT fleet, these mechanisms can be combined with gateway-enforced policies (whitelists, device revocation lists, per-device keys) to ensure that compromise of one endpoint does not imply compromise of all devices.

4.5. Complementary Defenses: IDS and Anomaly Detection

E2E encryption protects confidentiality and integrity of measurements in transit and through untrusted brokers, but it does not detect malicious behavior from already-authorized devices or adversaries who exploit vulnerabilities in endpoints. Therefore, encryption should be treated as one layer in a broader architecture that can include gateway-level anomaly detection and IDS. TinyML-based approaches on edge devices demonstrate the feasibility of local inference for authentication or anomaly detection under constrained resources [58]. Such edge deployments commonly rely on Wi-Fi-based embedded hardware platforms in practical smart-home and IIoT settings, reinforcing the relevance of the low-cost ESP-class substrate considered in this work [59]. Recent work specifically addressing evasion attacks against IoT-based IDS emphasizes that detection systems must also be robust to adversarial manipulation and may require adaptive defenses [56]. This supports a layered security posture: E2E encryption prevents broker and network tampering, while IDS and analytics help detect compromised endpoints and operational anomalies.

4.6. Limitations and Future Work

This study intentionally focuses on a practical baseline with a single ESP32 and Raspberry Pi pair, a low-rate sensor payload, and one sustained message rate (5 s interval) over a one-hour test. Additional experiments across different payload sizes, sampling intervals, and multiple concurrent clients are needed to quantify scalability and broker contention effects, especially at higher QoS levels where broker overhead can increase [8–10]. The latency

measurements report average/min/max; future work should log full distributions, compute standard deviations and confidence intervals, and incorporate energy or CPU utilization measurements to strengthen cross-paper comparability, particularly when contrasting AES-GCM with lightweight AEAD alternatives such as ASCON [20,21]. Finally, the key provisioning method is simplified (PSK), and nonce management is discussed but not exhaustively stress-tested under long-term reboot/counter edge cases. Addressing secure provisioning, rotation, and revocation at scale—potentially via secure elements, ECC, or PUF-bound identities—remains an important direction for deployment-grade systems [36,52,55].

5. Conclusions

This work presented a practical, end-to-end encrypted data pipeline for Industrial IoT built from widely available components: an ESP32 client implemented in C++ with mbedTLS, an untrusted MQTT broker, and a Raspberry Pi gateway implemented in Python using PyCryptodome. The protocol employs AES-256-GCM with associated data to protect confidentiality, integrity, and authenticity at the application layer, treating the broker purely as a relay. Experimental results show that the ESP32 can prepare encrypted packets in 41.754 ms on average, with AES-GCM accounting for the dominant share of this latency, while JSON serialization and Base64 encoding remain smaller contributors. Functional tests demonstrate that tampered frames are reliably rejected with explicit MAC failure logs and that unauthorized devices are blocked by whitelist-based authorization. A one-hour soak test at one message every five seconds achieved a packet success rate of 99.72%, indicating that the system can sustain reliable encrypted acquisition on commodity hardware.

The central conclusion is that application-layer E2E encryption is technically and operationally feasible in typical IIoT monitoring regimes and meaningfully strengthens MQTT architectures by eliminating trust in the broker for message content. Limitations include the narrow benchmark scope (single device pair, one sustained sampling rate, short duration) and simplified key provisioning. Future work should expand scalability testing (payload sizes, message intervals, multiple clients), quantify energy/CPU impact, and integrate stronger fleet key management using secure elements, ECC, or PUF-bound identities.

Author Contributions: Conceptualization, S.I. and G.A.; methodology, S.I.; software, S.I. and A.A.; validation, S.A., M.Z. and S.C.; formal analysis, S.I. and S.C.; investigation, S.I., A.A. and S.A.; resources, G.A. and M.Z.; data curation, S.I.; writing—original draft preparation, S.I.; writing—review and editing, G.A., S.C. and M.Z.; visualization, S.I. and A.A.; supervision, G.A.; project administration, G.A. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Ministry of Education and Science of the Republic of Kazakhstan grant number BR24992975 “Development of a Digital Twin for the Food Industry Enterprise Using Artificial Intelligence and IIoT Technologies”.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Acknowledgments: The authors thank the open-source communities behind mbedTLS, PyCryptodome, Eclipse Paho, Mosquitto, and InfluxDB for their software and documentation.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Cruz, C.; Palomar, E.; Bravo, I.; Gardel, A. Cooperative Demand Response Framework for a Smart Community Targeting Renewables: Testbed Implementation and Performance Evaluation. *Energies* **2020**, *13*, 2910. [\[CrossRef\]](#)
2. Herrera, V.A.S.; de Araújo, H.P.; Penteado, C.G.; Gazziro, M.; Carmo, J.P. Low-Cost Embedded System Applications for Smart Cities. *Big Data Cogn. Comput.* **2025**, *9*, 19. [\[CrossRef\]](#)
3. Stadler, S.; Borrero, E.R.; Zauner, J.; Hanshans, C. Development and Implementation of an Open Source IoT Platform, Network and Data Warehouse for Privacy-Compliant Applications in Research and Industry. *Curr. Dir. Biomed. Eng.* **2021**, *7*, 507–510. [\[CrossRef\]](#)
4. Lin, M.-H.; Wu, S.-H.; Huang, B.-W.; Chen, P.-H.; Huang, C.-H.; Chen, C.-Y.; Yang, C.-F. Node-RED Web-based Monitor and Control of Power System Using Modbus and Message Queuing Telemetry Transport Communication in Raspberry Pi Embedded Platform. *Sens. Mater.* **2024**, *36*, 4849–4864. [\[CrossRef\]](#)
5. Jornet-Monteverde, J.A.; Galiana-Merino, J.J. Low-Cost Conversion of Single-Zone HVAC Systems to Multi-Zone Control Systems Using Low-Power Wireless Sensor Networks. *Sensors* **2020**, *20*, 3611. [\[CrossRef\]](#) [\[PubMed\]](#)
6. Rouillard, J.; Vannobel, J.-M. Multimodal Interaction for Cobot Using MQTT. *Multimodal Technol. Interact.* **2023**, *7*, 78. [\[CrossRef\]](#)
7. Zhu, Z.; Jiao, T.; Li, Z. Innovative Applications of IoT in Smart Home Systems: Enhancing Environmental Monitoring with Integrated Sensor Technologies and MQTT Protocol. *JoWUA* **2024**, *15*, 69–89. [\[CrossRef\]](#)
8. Patel, J.; Gamess, E. A Study of the Resilience of the Mosquitto MQTT Broker Running on Raspberry Pi Against Fuzzy DDoS Attacks. In Proceedings of the 2024 Latin America Networking Conference, Bahia Blanca, Argentina, 15–16 August 2024. [\[CrossRef\]](#)
9. Dikii, D. Denial-of-Service Attack Analysis by MQTT Protocol. *Sci. Tech. J. Inf. Technol. Mech. Opt.* **2020**, *20*, 223–232. [\[CrossRef\]](#)
10. Ford, T.N.; Gamess, E.; Ogden, C. Performance Evaluation of Different Raspberry Pi Models as MQTT Servers and Clients. *IJCNC* **2022**, *14*, 1–18. [\[CrossRef\]](#)
11. Gentile, A.F.; Macrì, D.; Carnì, D.L.; Greco, E.; Lamonaca, F. A Performance Analysis of Security Protocols for Distributed Measurement Systems Based on IoT with Constrained Hardware and Open Source Infrastructures. *Sensors* **2024**, *24*, 2781. [\[CrossRef\]](#)
12. Gavriilidis, N.O.; Halkidis, S.T.; Petridou, S. Empirical Evaluation of TLS-Enhanced MQTT on IoT Devices for V2X Use Cases. *Appl. Sci.* **2025**, *15*, 8398. [\[CrossRef\]](#)
13. Machado, R.; Pinheiro, L.; Santos, V.; Salgado, B. PlugID: A Platform for Authenticated Energy Consumption to Enhance Accountability and Efficiency in Smart Buildings. *Energies* **2025**, *18*, 5466. [\[CrossRef\]](#)
14. Hernández-Gutiérrez, C.A.; Delgado-Del-Carpio, M.; Zebadúa-Chavarría, L.A.; Hernández-De-León, H.R.; Escobar-Gómez, E.N.; Quevedo-López, M. IoT-Enabled System for Detection, Monitoring, and Tracking of Nuclear Materials. *Electronics* **2023**, *12*, 3042. [\[CrossRef\]](#)
15. Guillén-Fernández, O.; Tlelo-Cuautle, E.; de la Fraga, L.G.; Sandoval-Ibarra, Y.; Nuñez-Perez, J.-C. An Image Encryption Scheme Synchronizing Optimized Chaotic Systems Implemented on Raspberry Pis. *Mathematics* **2022**, *10*, 1907. [\[CrossRef\]](#)
16. Chang, C.-C.; Lee, Y.-Y.; Hou, T.-Y.; Yu, C.-C. Information Security in Wireless Water Flow and Leakage Alarm System. *Sens. Mater.* **2022**, *34*, 2189–2197. [\[CrossRef\]](#)
17. Zhang, H.; Zhang, R.; Sun, J. Developing Real-Time IoT-Based Public Safety Alert and Emergency Response Systems. *Sci. Rep.* **2025**, *15*, 29056. [\[CrossRef\]](#)
18. Cruz, G.R.; Rojas-Cortés, J.E.; Cuaya-Simbro, G.; Simón-Marmolejo, I. Fleet Control with IoT Using TLS Certificates and SIM7000G GPS Device. *J. Appl. Eng. Technol. Sci.* **2024**, *6*, 754–766. [\[CrossRef\]](#)
19. Silva, C.; Cunha, V.A.; Barraca, J.P.; Aguiar, R.L. Analysis of the Cryptographic Algorithms in IoT Communications. *Inf. Syst. Front.* **2023**, *26*, 1243–1260. [\[CrossRef\]](#)
20. El-Hajj, M.; Gebremariam, T.H. Enhancing Resilience in Digital Twins: ASCON-Based Security Solutions for Industry 4.0. *Network* **2024**, *4*, 260–294. [\[CrossRef\]](#)
21. Sarasa Laborda, V.; Hernández-Álvarez, L.; Hernández Encinas, L.; Sánchez García, J.I.; Queiruga-Dios, A. Study About the Performance of Ascon in Arduino Devices. *Appl. Sci.* **2025**, *15*, 4071. [\[CrossRef\]](#)
22. Saleh, S.S.; Al-Awamry, A.A.; Taha, A. Tailoring AES for Resource-Constrained IoT Devices. *Indones. J. Electr. Comput. Eng.* **2024**, *36*, 290–301. [\[CrossRef\]](#)
23. Naser, H.Y.; Mattar, A.K.; Saare, M.A.; Almaiah, M.A.; Shehab, R. A Comparison of Lightweight Cryptographic Protocols for Energy-Efficient and Sustainable IoMT Authentication. *Eng. Technol. Appl. Sci. Res.* **2025**, *15*, 25746–25756. [\[CrossRef\]](#)
24. Shete, R.P.; Bongale, A.M.; Dharrao, D. Lightweight Cryptographic and Scalable IoT Systems for Encryption across GSM-MQTT Architectures in Resource-Constrained Aquaculture Environment. *Eng. Technol. Appl. Sci. Res.* **2025**, *15*, 25133–25139. [\[CrossRef\]](#)
25. Nagy, N.; Alnemer, S.; Alshuhail, L.M.; Alobiad, H.; Almulla, T.; Alrumaihi, F.A.; Ghadra, N.; Nagy, M. Module-Lattice-Based Key-Encapsulation Mechanism Performance Measurements. *Sci* **2025**, *7*, 91. [\[CrossRef\]](#)
26. Satrya, G.B.; Agus, Y.M.; Ben Mnaouer, A. A Comparative Study of Post-Quantum Cryptographic Algorithm Implementations for Secure and Efficient Energy Systems Monitoring. *Electronics* **2023**, *12*, 3824. [\[CrossRef\]](#)
27. Khan, U.H.; Qamar, A.; Khan, R.; Alturise, F.; Alshaabani, A.R.; Alkhalaf, S. Secure Edge-Based IoMT Framework for ICU Monitoring with TinyML and Post-Quantum Cryptography. *Sci. Rep.* **2025**, *15*, 1–23. [\[CrossRef\]](#) [\[PubMed\]](#)

28. Kumar, A.; Singh, P.; Kamble, D.P.; Singh, I. Hybrid Cryptographic Approach for Strengthening IoT and 5G/B5G Network Security. *Sci. Rep.* **2025**, *15*, 36195. [[CrossRef](#)] [[PubMed](#)]
29. Öztürk, G.; Çimen, M.E.; Çavuşoğlu, Ü.; Eldoğan, O.; Karayel, D. Secure and Efficient Data Encryption for Internet of Robotic Things via Chaos-Based Ascon. *Appl. Sci.* **2025**, *15*, 10641. [[CrossRef](#)]
30. A J, B.; Kaythry, P. Secure IoV Communications for Smart Fleet Systems Empowered with ASCON. *Sci. Rep.* **2025**, *15*, 19103. [[CrossRef](#)]
31. Wannaboon, C.; Ridzwan, S.A.B.S.; Fong-In, S. Chaotic Compressed Sensing for Secure Image Transmission in LoRa IoT Systems. *IJACSA* **2025**, *16*, 942–949. [[CrossRef](#)]
32. Hussein, N.A.; Shujaa, M.I. DNA Computing-Based Stream Cipher for Internet of Things Using MQTT Protocol. *IJECE* **2020**, *10*, 1035–1042. [[CrossRef](#)]
33. Seedorf, J.; Rawal, D.; Hamann, M.; Seid, S.; Schneider, K.; Anh, V.D.; Haag, M.; Milla, F.; Altun, Y.E. On-Sensor Stream Cipher Encryption for Protecting Smart City Sensor Data Directly on Resource-Constrained IoT Sensors. *ISPRS Arch.* **2025**, *48*, 105–112. [[CrossRef](#)]
34. Pearson, B.; Luo, L.; Zou, C.; Crain, J.; Jin, Y.; Fu, X. Building a Low-Cost and State-of-the-Art IoT Security Hands-On Laboratory. In Proceedings of the Second IFIP International Cross-Domain Conference, IFIP IoT 2019, Tampa, FL, USA, 31 October–1 November 2019. [[CrossRef](#)]
35. Ala-Laurinaho, R.; Autiosalo, J.; Tammi, K. Open Sensor Manager for IIoT. *J. Sens. Actuator Netw.* **2020**, *9*, 30. [[CrossRef](#)]
36. Díaz, A.F.; Blokhin, I.; Anguita, M.; Ortega, J.; Escobar, J.J. Multiprotocol Authentication Device for HPC and Cloud Environments Based on Elliptic Curve Cryptography. *Electronics* **2020**, *9*, 1148. [[CrossRef](#)]
37. Microchip Technology Inc. *AVR-IoT WG Development Board User Guide*; Microchip Technology Inc.: Chandler, AZ, USA, 2018.
38. Abdalah, R.W.; Abdulateef, O.F.; Hamad, A.H. A Centralized Federated Learning Algorithm Based Multi Classification Predictive Maintenance in IIoT System. *J. Eur. Systèmes Autom.* **2025**, *58*, 1123–1133. [[CrossRef](#)]
39. Sowmiya, K.; Anitha, V. A Context-Aware IoT and Edge Computing Framework for Wireless Plant Disease Diagnosis Using Compressed MaskRCNN and ResNet-50. *JoWUA* **2025**, *16*, 707–720. [[CrossRef](#)]
40. Khan, T. A Deep Learning-Based Gunshot Detection IoT System with Enhanced Security Features and Testing Using Blank Guns. *IoT* **2025**, *6*, 5. [[CrossRef](#)]
41. Villafuerte, N.; Manzano, S.; Ayala, P.; García, M.V. Artificial Intelligence in Virtual Telemedicine Triage: A Respiratory Infection Diagnosis Tool with Electronic Measuring Device. *Future Internet* **2023**, *15*, 227. [[CrossRef](#)]
42. De Side, G.N.; Putra, G.M.D.; Setiawati, D.A. IoT Microclimate Monitoring System Using Node-RED Platform at Plant Factory. In *IOP Conference Series: Earth and Environmental Science*; IOP Publishing: Bristol, UK, 2025. [[CrossRef](#)]
43. Sebti, M.R.; Dakhia, Z.; Carabetta, S.; Di Sanzo, R.; Russo, M.; Merenda, M. Real-Time Classification of Ochratoxin A Contamination in Grapes Using AI-Enhanced IoT. *Sensors* **2025**, *25*, 784. [[CrossRef](#)]
44. Conceição, G.; Coelho, T.; Mota, A.; Briga-Sá, A.; Valente, A. Smart Matter-Enabled Air Vents for Trombe Wall Automation and Control. *Electronics* **2025**, *14*, 3741. [[CrossRef](#)]
45. Khan, S.; Nazir, S.; Khan, H.U. Smart Object Detection and Home Appliances Control System in Smart Cities. *Comput. Mater. Contin.* **2021**, *67*, 895–915. [[CrossRef](#)]
46. Salikhov, R.B.; Abdrakhmanov, V.K.; Safargalin, I.N. Internet of Things (IoT) Security Alarms on ESP32-CAM. *J. Phys. Conf. Ser.* **2021**, *2096*, 012109. [[CrossRef](#)]
47. Hussein, N.A.; Shujaa, M.I. Secure Vehicle-to-Vehicle Voice Chat Based on MQTT and CoAP IoT Protocol. *Indones. J. Electr. Eng. Comput. Sci.* **2020**, *19*, 526–534. [[CrossRef](#)]
48. Rashid, H.A.; Chowdhury, O.; Hossain, M.M.; Rahman, M.M.; Muhammad, G.; AlQahtani, S.A.; Alrashoud, M.; Yassine, A.; Hossain, M.S. IoT-Based Medical Image Monitoring System Using HL7 in a Hospital Database. *Healthcare* **2023**, *11*, 139. [[CrossRef](#)]
49. Serepas, F.; Papias, I.; Christakis, K.; Dimitropoulos, N.; Marinakis, V. Lightweight Embedded IoT Gateway for Smart Homes Based on an ESP32 Microcontroller. *Computers* **2025**, *14*, 391. [[CrossRef](#)]
50. Arcenegui, J.; Arjona, R.; Román, R.; Baturone, I. Secure Combination of IoT and Blockchain by Physically Binding IoT Devices to Smart NFTs Using PUFs. *Sensors* **2021**, *21*, 3119. [[CrossRef](#)] [[PubMed](#)]
51. Atanov, S.; Seitkulov, Y.; Moldamurat, K.; Yergaliyeva, B.; Kyzzyrkanov, A.; Seitbattalov, Z. About One Lightweight Encryption Algorithm Ensuring the Security of Data Transmission and Communication Between IoT Devices. *Int. J. Electr. Comput. Eng.* **2024**, *14*, 6861. [[CrossRef](#)]
52. Gupta, A.; Surpur, A.; Das, B.P.; Manhas, S. A Unified Approach to a Secure and Lightweight Mutual Authentication Protocol Using Pre-Characterized COTS SRAM ICs for IoT Applications. *ACM Trans. Embed. Comput. Syst.* **2025**, *24*, 1–27. [[CrossRef](#)]
53. Pérez, E.; Araiza, J.C.; Pozos, D.; Bonilla, E.; Hernández, J.C.; Cortes, J.A. Application for Functionality and Registration in the Cloud of a Microcontroller Development Board for IoT in AWS. *Appl. Comput. Sci.* **2021**, *17*, 14–27. [[CrossRef](#)]
54. Chiu, J.-C.; Lee, G.-Y.; Hsieh, C.-Y.; Lin, Q.-Y. Design and Implementation of Nursing-Secure-Care System with mmWave Radar by YOLO-v4. *Appl. Syst. Innov.* **2024**, *7*, 10. [[CrossRef](#)]

55. Jiteurtragool, N.; Samkunta, J.; Ketthong, P. Lightweight Parabola Chaotic Keyed Hash Using SRAM-PUF for IoT Authentication. *IJACSA* **2025**, *16*, 724. [[CrossRef](#)]
56. Abu Laila, D. Responsive Machine Learning Framework and Lightweight Utensil of Prevention of Evasion Attacks in the IoT-Based IDS. *STAP J. Secur. Risk Manag.* **2025**, *2025*, 59–70. [[CrossRef](#)]
57. Liu, T. *Digital-Output Relative Humidity & Temperature Sensor/Module DHT22 (AM2302)*; Aosong Electronics Co., Ltd.: Guangzhou, China, 2010.
58. Thota, Y.R.; Nixon, J.S.; Chandran, B.; Nikoubin, T. TinyML-Based Biometric Authentication Using PPG Signals for Edge Devices. In Proceedings of the Great Lakes Symposium on VLSI 2025, New Orleans, LA, USA, 30 June–2 July 2025. [[CrossRef](#)]
59. Hercog, D.; Sedonja, D.; Recek, B.; Truntič, M.; Gergič, B. Smart Home Solutions Using Wi-Fi-Based Hardware. *Teh. Vjesn.* **2020**, *27*, 1351–1358. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.